⚡ AeroMetrics Documentation

Welcome to the production deployment guide and technical reference index for the **AeroMetrics Enterprise API Observability Engine**. This document outlines installation sequences, core architectures, real-time verification benchmarks, and optimization strategies.

🛠️ Core Architectural Highlights

Thread-Safe SQLite Pooling Context (database.py): Utilizes a synchronization-locked queue-backed connection manager enforcing Write-Ahead Logging (`PRAGMA journal_mode=WAL;`) to handle parallel reads and sequential worker writes concurrently without data contention or lock bottlenecks.

Statistical Outlier Filters (engine.py): Features an anomaly detection engine that tracks a moving historical data baseline (past 30 dispatches) utilizing a rolling **Z-score metric** (where $Z > 2.5$) to instantly isolate and flag real-time performance latency drift.

Automated Project Bootstrapper (run.py): Requires zero manual directory or file path provisioning. On initial startup, the core layer dynamically verifies, builds, and scales out application storage folders (`logs/` , `exports/`), indexes the tracking matrix schema database, and seeds administrative keys automatically.

Decoupled Processing Workflows (scheduler.py & reporting.py): Offloads runtime performance polling completely out of the active Flask client execution frame using daemonized background threads (`APScheduler`) and compiles multi-format metrics exports to CSV, clean structured Excel workbooks, or high-fidelity executive PDFs.

📂 System Project Footprint

```
api_monitor/
│
├── run.py                  # Application Entry Point & Auto-Bootstrapper
├── config.py               # Configuration Global Environment Variables
├── database.py             # Thread-Safe SQLite Connection Pooler & Indices
├── auth.py                 # Session-Based Access Middleware Gatekeeper
├── engine.py               # Advanced Performance Monitor Worker & Outlier Detection
├── scheduler.py            # Background APScheduler Worker Orchestrator
├── reporting.py            # Report Exporter Component (CSV, Excel, PDF)
│
├── logs/                   # System Runtime Tracking Buffers (Auto-created)
│   ├── monitor.log
│   └── app.log
│
├── exports/                # Generated On-Demand Management Reports (Auto-created)
│
└── templates/              # UI Presentation Template Layouts
    ├── base.html           # Global Design Template Frame
    ├── login.html          # Portal Administrative Login UI
    ├── dashboard.html      # Interactive Metrics & Anomaly Interface
    └── manage_apis.html    # System Endpoints Administration Panel
```

🚀 Installation & Rapid Deployment Guide

1. Provision Platform Environment Dependencies

Ensure your local Python path execution pipeline is mapped correctly and install core dependencies:

```
python -m pip install Flask requests apscheduler pandas openpyxl reportlab
```

2. Launch the Control Server

Run the bootstrapper directly from the terminal console. The engine will dynamically build storage paths and launch the interface:

```
python run.py
```

3. Authenticate Access

- **URL Link Target:** `http://127.0.0.1:5000/`
- **Default Operator Handle:** `admin`
- **Default Operator Secret Key:** `admin123`

🧪 Live Telemetry Field Verification Framework

To test your premium layout metrics graphs and validation warnings instantly, add these verified public endpoints inside the **Node Manager** panel:

TARGET SCOPE PROFILE	ABSOLUTE TARGET URL	METHOD	CYCLE FRAME
Production Gateway Ping	`https://jsonplaceholder.typicode.com/todos/1`	GET	10 seconds
Upstream Auth Microservice	`https://reqres.in/api/users?delay=1`	GET	15 seconds
Legacy Profile Endpoint	`https://reqres.in/api/unknown/23`	GET	20 seconds

⚡ How to Trigger the Live Statistical Anomaly Warning:

1. Add **Upstream Auth Microservice** (`https://reqres.in/api/users?delay=1`) and allow it to cycle roughly 10 consecutive times until its moving window baseline stabilizes around 1000ms.
2. Click **Scrub Node** to wipe out its target record context from active configurations.
3. Immediately re-add the endpoint with the exact same identity name, but swap out the delay parameter: `https://reqres.in/api/users?delay=4` .
4. On the very next check cycle execution, the **Moving Z-Score algorithmic filter** will notice the sudden spike from 1s to 4s, identify it as a major data variant, and trigger the red **"Statistical Latency Outlier Triggered"** warning bar on the interface.

💼 Enterprise Cloud Environments Hardening

- **Enforce Cryptographic Tokens:** Change `Config.SECRET_KEY` inside `config.py` to a long random hexadecimal hash signature or read it out of your system environment flags (`os.environ.get`).

- **WSGI Server Wrapper Integration:** Avoid running the integrated python runtime engine loop (`app.run()`) directly in production. Run the platform scaling workers securely behind a dedicated production proxy layer such as `gunicorn` :

```
gunicorn --workers 4 --threads 2 --bind 0.0.0.0:8000 run:app
```